

2022141461109-殷浩杨-第一次课后作业

一、项目概况与背景

1.1 系统定位

yolink-front（悦联岛）是一个面向在校学生群体的校园社交平台前端应用。其核心价值在于打通校园场景下"内容创作——社区互动——个人展示"三条主线：用户可以通过富文本编辑器创作并发布校园动态，在信息流中浏览和评论他人的帖子，亦可管理个人主页与账号设置。整个应用以移动优先的响应式设计呈现，兼顾桌面端的阅读与创作体验。

从技术选型来看，本项目采用 React 19 与 TypeScript 作为核心框架，以 TanStack Router 实现类型安全的客户端路由，以 TanStack Query（React Query）统一管理服务端状态与数据缓存，构建工具使用 Vite。这一技术组合代表了 2025 年前端工程领域的主流实践，具备较高的工程化成熟度。

1.2 核心功能模块

功能模块	目录路径	核心职责
认证（auth）	src/features/auth/	用户登录、会话持久化
首页信息流（home）	src/features/home/	校园动态聚合、标签浏览
帖子详情（post-detail）	src/features/post-detail/	帖子渲染、评论区交互
富文本编辑器（editor）	src/features/editor/	基于 Tiptap 的内容创作
个人中心（profile）	src/features/profile/	用户主页展示与资料编辑
搜索（search）	src/features/search/	全文检索与结果渲染
通知（notifications）	src/features/notifications/	系统通知推送展示
设置（settings）	src/features/settings/	账号偏好与隐私管理
公告（announcement）	src/features/announcement/	站内公告内容详情

1.3 独立前端仓库的工程背景

需要特别说明的是，yolink-front 以**独立前端仓库**的形式进行开发与维护，与后端服务仓库完全解耦。这一架构决策并非偶然，而是经过深思熟虑的工程选择：前端通过约定的 REST API 契约与后端通信，部署流程亦相互独立。这一背景直接决定了本项目在定义软件过程时的职责边界——前端不承担数据库设计、系统运维等后端关切，亦无法独立控制系统级的端到端集成。这是我们后续进行过程剪裁的重要前提。

二、基于 ISO/IEC 12207 的软件过程定义

在通读课程分发的 ISO/IEC 12207:2017 《系统与软件工程——软件生命周期过程》全文后，笔者依据该标准的三层过程体系——**主要过程（Primary Processes）、支持过程（Supporting Processes）与组织过程（Organizational Processes）**——结合 yolink-front 的实际开发活动，逐一进行定义并映射如下。

2.1 主要过程

主要过程是直接服务于软件系统获取、开发与运行的核心活动，对应 ISO/IEC 12207 第 6.3 至 6.4 章节的技术过程群。

2.1.1 需求分析过程 → 功能路由设计

在传统软件工程范式中，需求分析的产出物是一份严格的软件需求规格说明（SRS）。然而，对于本项目而言，需求分析活动的直接输出是 TanStack Router 的**路由树设计**。每一条路由（如 `/profile/$profileId`、`/post/$postId`）代表一个用户可到达的功能入口，对应一组明确的用户诉求。`src/routes/` 目录下的路由文件，以及由 `routeTree.gen.ts` 自动生成的路由注册表，事实上构成了一份活的、可执行的“功能清单”，其信息密度远超静态的 SRS 文本。

此外，各功能模块下的 `types/` 目录（如 `src/features/editor/types/`、`src/features/post-detail/types/`）以 TypeScript 接口与类型别名的形式，将业务实体的数据结构精确描述出来，弥补了路由层在数据契约上的缺失。这一组合共同承担了“需求分析 → 设计规格”的过程职责。

2.1.2 体系结构设计过程 → 领域驱动分层架构

在体系结构层面，本项目采用以 `src/features/` 为核心的**领域驱动分层结构**。每个功能领域（如 `home`、`profile`、`editor`）内部均遵循统一的四层分层约定：

- `api/` 层：封装 Axios 请求调用，将 HTTP 细节与业务逻辑隔离；

- **hooks/ 层**：通过 TanStack Query 的 `useQuery` / `useMutation` 封装，将数据获取逻辑与 UI 渲染解耦；
- **components/ 层**：实现具体的 UI 组件，消费 hooks 层提供的数据；
- **pages/ 层**：组合组件形成完整页面，与路由系统对接。

全局共享的基础设施由 `src/components/ui/`（shadcn/ui 组件库）、`src/lib/`（工具函数）和 `src/components/layout/`（布局骨架）提供。这一结构在首次接触时可能显得繁琐，但在项目规模扩大后，其价值体现在：任意一个功能模块的修改都不会意外影响其他模块，这正是 ISO/IEC 12207 体系结构设计过程所追求的**关注点分离（Separation of Concerns）**目标的具体实现。

2.1.3 软件构造过程 → TypeScript 编码实践

软件构造过程的执行贯穿整个开发周期。在 TypeScript 的约束下，每一次函数调用、每一个组件的 Props 传递，都经过编译时的类型检验。以

`src/features/editor/types/CreatePostRequest.ts` 中定义的请求体类型为例，它既是编辑器组件向 API 层传参的契约，也是后端接口联调时双方对齐数据结构的依据——一份类型定义，同时履行了"设计文档"与"接口契约"的双重职责。

2.1.4 软件集成过程 → 前端内部模块集成

在构造完成后，软件集成过程的重点在于验证各模块之间的协作正确性。对于本项目而言，这主要体现在：TanStack Router 的路由跳转是否正确传递了路径参数（如 `$postId`、`$profileId`），全局布局组件 `AppLayout.tsx` 是否在所有页面中正常渲染，以及 `AppSidebar.tsx` 中的导航状态是否与当前路由保持同步。这一集成验证通过本地开发环境的手动测试完成。关于系统级集成过程的剪裁，详见第四节。

2.2 支持过程

支持过程是为主要过程提供保障与服务的辅助活动，对应 ISO/IEC 12207 第 6.2 章节。

2.2.1 质量保证过程 → ESLint 与 TypeScript 编译检查

本项目在质量保证方面采用了**工具链驱动**的策略，而非依赖人工评审文档。首先，TypeScript 编译器在每次 `vite build` 时作为静态分析器运行，任何类型不一致都会导致构建失败，从根本上阻断了类型错误流入代码库的通道。此外，`eslint.config.js` 中配置的 ESLint 规则集（包含 React Hooks 规则等）在编辑器层面实时提示潜在问题，将质

量反馈环路压缩至"编写代码"阶段。这两道工具链构成了本项目质量保证的第一道防线，其有效性并不依赖于团队规模的大小。

2.2.2 配置管理过程 → Git 版本控制

本项目以 Git 作为配置管理工具。主分支（`main`）始终维持在可运行状态，所有新功能开发在独立的功能分支上进行，完成后通过合并请求（Pull Request）并入主分支。这一工作流不仅提供了完整的变更历史追溯，也使得在引入缺陷时能够快速定位到具体的代码提交，满足 ISO/IEC 12207 配置管理过程对"基线标识"与"变更控制"的要求。

2.2.3 文档管理过程 → 敏捷文档策略

本项目对文档管理过程进行了显著剪裁，以"敏捷文档"策略替代传统的形式化文档体系。关于这一剪裁的深度思考，详见第四节第 4.1 部分。

2.2.4 问题解决过程 → Issue 跟踪与迭代修复

在开发过程中发现的缺陷通过 Git 仓库的 Issue 功能记录，并按照严重程度纳入后续迭代的修复计划。这一闭环机制确保了没有已知问题被遗忘，同时避免了因紧急修复而打乱当前迭代节奏。

2.3 组织过程

组织过程为整个项目的运作提供顶层框架，对应 ISO/IEC 12207 第 6.1 至 6.3 章节中的项目管理与组织基础设施过程。

2.3.1 项目规划过程 → 迭代任务拆解

在每轮迭代启动之前，笔者将当前阶段的用户故事拆分为粒度适当的开发任务，估算各任务的时间成本，并据此设定本轮迭代的完成目标（Definition of Done）。这一规划过程轻量但不缺席，既保证了开发方向的明确性，又为后续的进度评估提供了对照基线。

2.3.2 风险管理过程 → 依赖风险应对

本项目识别了两类主要技术风险：其一是第三方库 API 的非兼容性变更（如 TanStack Router v1 至 v2 的迁移成本），应对策略为在 `package.json` 中锁定依赖版本；其二是后端接口的不稳定性（联调初期接口频繁调整），应对策略为在 `src/api/axiosInstance.ts` 中统一封装请求基础配置，使得接口地址变更的影响范围最小化。

三、过程来源与能力评估思路

3.1 主要参考来源：ISO/IEC 12207:2017

本报告所定义的全部软件过程均以课程分发的 **ISO/IEC 12207:2017** 文档为直接参考来源。该标准提供了一套完备的软件生命周期过程分类框架，其过程命名、活动描述与工作产品定义构成了本报告映射工作的基准坐标系。在阅读标准原文的过程中，笔者深切体会到 ISO/IEC 12207 的设计初衷是面向“具备一定组织规模与合同约束的软件项目”——这一认知，正是后续引入过程评估视角、触发过程剪裁思考的起点。

3.2 辅助参考：ISO/IEC 33001 与过程能力评估

在过程评估维度，笔者借鉴了 **ISO/IEC 33001:2015《信息技术——过程评估——概念与术语》** 所定义的过程能力等级模型（Level 0 至 Level 5）。基于此模型对本项目的现状进行审视后，笔者得出以下判断：

首先，若不加剪裁地将 ISO/IEC 12207 的全部过程要求照搬到本项目中，势必要求在每个阶段编制大量正式文档（SRS、SDD、测试报告等），并经过正式评审。然而，这些活动在独立开发的前端项目中所能产生的价值，远低于其所消耗的时间成本。基于 ISO/IEC 33001 的能力等级视角来看，这种“过程定义了但实际上无法执行”的状态，在标准中被明确界定为 **Level 0（不完整，Incomplete）**——定义再完美的过程，若无法转化为真实活动与可见产出，其实际能力等级为零。

此外，ISO/IEC 33001 强调，**Level 1（已执行，Performed）** 的核心判据是：过程活动确实发生，且产出了可标识的工作产品。基于此，本项目将过程能力目标务实地设定在 Level 1，即确保每一个保留的过程都有实际执行记录与对应产出物，而非追求形式上更高的能力等级却在执行层面形同虚设。

这一来自 ISO/IEC 33001 的“过程评估”视角，为本项目的过程剪裁提供了理论依据：**剪裁的本质不是降低质量标准，而是将过程能力的实现从“纸面合规”转向“真实执行”。**

四、过程剪裁的深度思考

ISO/IEC 12207 第 4.7 条明确赋予组织对标准进行**剪裁（Tailoring）**的权利，要求剪裁行为有充分的理由支撑，且须得到相关利益相关方的认可并形成记录。本项目针对以下三个方面进行了经过深思熟虑的过程剪裁。

4.1 文档剪裁：从"重文档"转向"敏捷文档"

剪裁内容：不单独编制软件需求规格说明（SRS）、软件设计说明（SDD）、测试计划（STP）等格式化文档，以 Markdown 笔记、代码注释与 TypeScript 类型定义作为主要文档载体。

思考过程：

在项目初期，笔者曾尝试按照传统规范为各功能模块编写详细的设计说明文档。然而，在经历了数次迭代后，一个令人沮丧的现象出现了：文档与代码开始出现**漂移（Drift）**——设计说明中描述的组件接口与实际代码中的 Props 定义已不再一致，而维护文档的时间成本远高于直接阅读代码。这一亲身经历促使笔者重新审视"文档"在本项目中的真实价值。

首先，React 的组件化开发模式本身具有极强的**自解释性**。以 `src/features/profile/` 为例，该模块下的 `api/` 目录封装了与个人主页相关的所有数据请求，`hooks/` 目录封装了 TanStack Query 的查询逻辑，`components/` 目录包含了纯粹的 UI 渲染逻辑。任何具备 React 基础的开发者，无需任何额外文档，即可通过目录结构和文件命名推断出各文件的职责。**组件边界即文档边界**。

此外，TypeScript 的类型系统提供了比任何文档更精确、更可信的"规格说明"。

`src/features/editor/types/CreatePostRequest.ts` 中定义的请求体类型，不仅描述了字段名称与数据类型，还通过 `?` 操作符明确了哪些字段是可选的——这是普通文档难以同等精确表达的语义信息，且这份"文档"与代码永远保持同步，因为它本身就是代码的一部分。

基于此，本项目采用**"敏捷文档"**策略：以 Markdown 文件（存放于 `docs/` 目录）记录架构决策与过程定义等高层次内容，以 TypeScript 类型定义记录数据契约，以代码注释记录非显而易见的业务逻辑。这一策略与敏捷宣言**"可工作的软件高于详尽的文档"**的价值观高度契合，同时并不意味着"零文档"——我们仍然保留了对**真正需要文字才能表达清楚的内容**的文档化义务。

4.2 集成剪裁：独立前端仓库下的局部验证策略

剪裁内容：不独立建立系统级集成测试（System Integration Testing）规程，不要求前端仓库产出系统集成测试报告，将集成验证范围限定于前端内部模块集成与接口契约联调。

思考过程：

ISO/IEC 12207 第 6.4.9 条"系统集成过程"所描述的活动，预设了一个包含多个子系统——硬件、操作系统、中间件、数据库、后端服务、前端客户端——的完整系统交付场

景，其目的是验证所有子系统按照系统需求规格协同工作。这一过程的价值毋庸置疑，但其前提是**由一个有权协调所有子系统的组织角色**来主导执行。

然而，yolink-front 是一个独立的前端仓库。它无法启动后端数据库，无法控制服务端的部署版本，也无法独立模拟真实的网络负载。在此约束下，若强行要求前端仓库承担"系统集成测试"的职责，不仅超出了仓库的工程边界，也会因测试环境的缺失而导致测试结果缺乏可信度。

取而代之，本项目通过两种更为务实的手段覆盖集成风险：其一，在本地开发阶段使用 **Mock Data** 模拟后端响应，对前端各模块在各种数据场景下的表现进行充分的局部验证——`src/api/axiosInstance.ts` 中的 `baseUrl` 配置使得切换 Mock 环境与真实环境仅需修改一处；其二，在联调阶段通过访问后端沙箱服务进行**接口契约验证**，确认前后端在数据格式与状态码处理上的一致性。这两种手段相结合，在独立前端仓库的边界内实现了对集成风险的有效管控。

综上所述，这一剪裁并非对集成测试责任的逃避，而是在正确的职责边界内选择了正确的验证手段。

4.3 合规性声明：教师认可下的过程改进行为

ISO/IEC 12207 第 4.7 条要求剪裁行为须经利益相关方认可。在本课程的教学场景下，**课程教师是本项目最重要的利益相关方**，其对开发方法论的立场构成剪裁合规性的直接依据。

教师在课程教学过程中明确表达了对敏捷开发方法的认可，指出敏捷方法在小型团队、高迭代频率、需求不确定性高的场景下具有显著的适用性优势，并明确允许学生项目在保证过程可追溯性的前提下采用敏捷实践替代传统的瀑布式过程。这一立场，为本报告中的所有剪裁行为提供了**组织层面的合规性背书**。

需要强调的是，本项目的剪裁是基于**"过程改进 (Process Improvement)"**逻辑的主动选择，而非因能力不足而被动省略。正如 ISO/IEC 33001 所揭示的：一个能力等级为 Level 1 但在每个保留过程上都真实执行的项目，其实际工程价值远高于定义了 Level 3 能力但从未真正执行的项目。

五、综合总结

综上所述，本报告以 ISO/IEC 12207:2017 为框架，结合 ISO/IEC 33001 的过程能力评估思想，为 yolink-front 校园社交平台前端项目构建了一套兼顾标准规范性与工程实用性的软件过程体系。

在过程定义层面，我们将标准的三层过程体系（主要过程、支持过程、组织过程）映射到 React 前端项目的具体开发活动中，以功能路由设计对应需求分析，以领域驱动分层架构对应体系结构设计，以工具链驱动的静态检查对应质量保证，以 Git 工作流对应配置管理。

在过程剪裁层面，我们基于以下三点做出了有据可查的剪裁决策：代码自解释性与 TypeScript 类型系统使"重文档"策略成本过高；独立前端仓库的工程边界使系统级集成测试规程不具可操作性；教师对敏捷实践的认可为上述剪裁提供了合规性保障。

最终，本项目的过程体系在执行层面可稳定达到 ISO/IEC 33001 所定义的 **Level 1（已执行）** 能力水平：每一个保留的过程均有真实的活动发生，均能产出可标识的工作产品，从而在有限的团队资源下，实现软件过程管理核心价值的最大化——**可追溯、可控制、可持续改进**。